

به نام خدا



دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس یادگیری تقویتی

تمرین سوم

نام و نام خانوادگی	امیر غرقابی
شماره دانشجویی	۸۱۰۱۰۲۲۱۷
تاریخ ارسال گزارش	۱۴۰۳.۰۹.۱۹

فهرست

۱. پاسخ ۱
۲. پاسخ ۳
۳. پاسخ ۴
۴. پیاده‌سازی ۶

شکل‌ها

- شکل ۱. روش اِپدیت کردن q-value در الگوریتم SARSA ۳
- شکل ۲. روش اِپدیت کردن q-value در الگوریتم Expected Sarsa ۳
- شکل ۳. مسیر safe و optimal در محیط cliff ۶
- شکل ۴. الگوریتم Q-learning ۷
- شکل ۵. نمودار reward و regret الگوریتم Q-learning با $\alpha = 0.1$ ثابت ۸
- شکل ۶. نمودار reward و regret الگوریتم Q-learning با α کاهشی ۹
- شکل ۷. مقایسه ی نتایج α ثابت و متغیر در الگوریتم Q-Learning ۹
- شکل ۸. نمودار reward و regret الگوریتم SARSA ۱۱
- شکل ۹. شمایی از الگوریتم N-step tree ۱۱
- شکل ۱۰. نمودار reward و regret الگوریتم one-step ۱۲
- شکل ۱۱. مقایسه الگوریتم های SARSA و one-step backup ۱۲
- شکل ۱۲. نمودار reward و regret الگوریتم two-step ۱۳
- شکل ۱۳. مقایسه الگوریتم های one step backup و two-step backup ۱۳
- شکل ۱۴. نمودار reward و regret الگوریتم three-step ۱۳
- شکل ۱۵. مقایسه الگوریتم های two step backup و three-step backup ۱۴
- شکل ۱۶. الگوریتم Off-policy monte carlo [5] ۱۵
- شکل ۱۷. نمودار reward و regret الگوریتم off-policy monte carlo در حالت ϵ کاهشی ۱۶
- شکل ۱۸. نمودار reward و regret الگوریتم off-policy monte carlo در حالت $\epsilon = 0.5$ ثابت ۱۶
- شکل ۱۹. نمودار reward و regret الگوریتم off-policy monte carlo در حالت $\epsilon = 0.1$ ثابت ۱۷

پاسخ ۱.

مسئله:

اگر reward ها تنک باشد، سرعت همگرایی کاهش میابد در این حالت، عامل برای explore کردن به مشکل میخورد. در این صورت ممکن است عامل در یک local optima گیر کند و نتواند s,a های بهینه را پیدا کند. در q learning لازم است که عامل تمام s,a ها را به تعداد کافی ببیند اما اگر reward ها تنک باشد، ممکن است این اتفاق نیافتد و در exploration-exploitation به مشکل برخورد کند.

اپدیت شدن q-value ها نیز واریانس بالایی خواهند داشت. زیرا یک reward نیز می تواند تاثیر بسیار زیادی بر اپدیت شدن q-value ها بگذارد و موجب ناپایداری یادگیری شود. درواقع اگر دنباله ی نسبتا بلندی از s,a ها برای رسیدن به یک reward باشد، می تواند واریانس q-value ها را بشدت بالا ببرد.

بنابراین برای یادگیری، نیازمند تعداد اپیزود بسیار بیشتری بوده و در نتیجه سرعت همگرایی کاهش میابد. [1]

راه حل:

برای حل مشکلات مطرح شده، نیازمند تغییراتی در روش explore و reard دهی به عامل ایجاد شود. بطور مثال از تکنیک های exploration استفاده شود تا در اپیزودهای مختلف، s,a های مختلف دیده شوند. برای این کار می توان از تکنیک epsilon-greedy استفاده کرد.

راه حل دیگر، استفاده از reward-shaping است. مثلا به عامل، در برخی از s,a ها، reward لحظه ای داده شود یعنی اگر در یک state، پاداش وجود دارد، در state های نزدیک به آن نیز پاداش ایجاد شود. می توان از imitation learning استفاده کرد. به این معنا که اگر بتوان s,a های یک عامل دیگر را دید، اگر عامل ما، رفتارهای آن عامل دیگر را تکرار کرد، پاداش دریافت کند. [2]

مشکلات مربوط به H.I.L.:

یکی از مشکلات، یعنی تنک بودن پاداش در بخش قبل بررسی شد.

مشکل دیگر، دینامیک پویا و متغیر انسان است. این مسئله موجب می شود که رفتار انسان ها در مواجهه با یک state، در تکرارهای مختلف، متفاوت باشد. راه حل این مسئله استفاده از imitation learning و social learning است. به این صورت که عامل، در مواجهه با یک محیطی که پاداش آن توسط انسان داده

می شود، نه تنها رفتار همان انسان بلکه رفتار انسان های دیگر را نیز در نظر گیرد. سعی کند Policy خود را با در نظر گرفتن آنها اپدیت کند.

مشکل دیگر، مسائل امنیتی و اخلاقی است. یعنی ممکن است که عامل، action هایی را انتخاب کند که موجب آسیب رساند به انسان شود. راه حل، استفاده از متد هایی مانند constrained RL است. به طوری که محدودیت های امنیتی به صورت مشخص، تعریف شده باشند.

رفتار انسان ها می تواند با یکدیگر متفاوت باشد بنابراین، عامل باید به صورت personalized (شخصی سازی شده) عمل کند. یعنی سیاست هر عامل، با توجه به preference های انسان های مختلف، تغییر کند. این امر موجب می شود که عامل نیازمند همواره یادگیری در طول زمان باشد.

امنیت دیتای انسان ها نیز مسئله است. راه حل آن، استفاده از مکانیزم هایی برای حفظ و نگهداری دیتا، مانند data encryption و روش هایی از این قبیل است. [3]

پاسخ ۲.

روش های SARSA و Expected SARSA در نحوه بروز رسانی Q با یکدیگر تفاوت دارند. در Sarsa، اکشن در $S=S'$ با در نظر گرفتن سیاست مشخص می شود. یعنی مثلا اگر سیاست، epsilon-greedy باشد، اکشن با بیشترین q-value با احتمال $1-\epsilon$ انتخاب شده و مابقی با احتمال $\frac{\epsilon}{|A|}$ انتخاب خواهند شد سپس با rewardی که دریافت می شود، q-value آپدیت می شود. (شکل ۱)

Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)
 $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

شکل ۱. روش آپدیت کردن q-value در الگوریتم SARSA

اما در Expected Sarsa، میانگینی از reward ها با در نظر گرفتن تمام اکشن ها در $S=S'$ محاسبه شده و این مقدار برای آپدیت کردن q-value ها استفاده خواهد شد. (شکل ۲)

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \mathbb{E}_{\pi}[Q(S_{t+1}, A_{t+1}) | S_{t+1}] - Q(S_t, A_t)] \\ &= Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \sum \pi(a|S_{t+1})Q(S_{t+1}, a) - Q(S_t, A_t)], \end{aligned}$$

شکل ۲. روش آپدیت کردن q-value در الگوریتم Expected Sarsa

روش Expected Sarsa به طور کلی پیچیدگی محاسباتی بیشتری دارد اما مقاومت بیشتری داشته و پایدارتر است. درواقع به این علت که احتمال و reward تمام اکشن های ممکن را در $S=S'$ در نظر میگیرد، واریانس بهتر (کمتر) دارد. اما در مقابل، در روش SARSA، به این علت که reward با توجه به انتخاب یک اکشن با توجه به سیاست، ایجاد می شود، در صورتی که سیاست مناسب نباشد، ممکن است موجب واریانس بالا شود و پایداری را از دست بدهد.

همچنین در یک مسئله ی مشخص، در نرخ همگرایی نیز Expected sarsa عملکرد بهتری خواهد داشت. درواقع Expected sarsa با در نظر گرفتن احتمال انتخاب تمام اکشن ها در $S=S'$ ، ارزیابی بهتری از Q-value داشته و آپدیت های سریعتری و پایداری خواهد داشت.

با اینکه Expected sarsa به طور کلی نسبت به sarsa از سرعت همگرایی و پایداری بیشتری برخوردار است، اما در برخی از شرایط، بهتر است که از SARSA استفاده شود.

بطور مثال اگر محدودیت های سخت افزاری موجب شود که مجبور به استفاده از sarsa شویم زیرا حجم محاسباتی کمتری دارد.

در محیط هایی که تعداد state ها محدود و state ها گسسته است، محاسبات نسبتاً پیچیده ی Expected sarsa ممکن است بازدهی بالایی نداشته باشد و استفاده از sarsa نیز با محاسبات کمتر، نتیجه بسیار خوبی را حاصل کند.

همچنین اگر تعداد اکشن ها نیز کم باشد، و اکشن ها گسسته باشند، نیز ممکن است سادگی محاسبات sarsa موجب شود که sarsa انتخاب مناسبتری نسبت به expected sarsa باشد.

درواقع انتخاب sarsa و یا expected sarsa وابسته به محیط و محدودیت های محاسباتی است. [4], [5]

پاسخ ۳.

مقایسه on-policy و off-policy از لحاظ:

بهره وری نمونه:

روش های on-policy از نظر بهره وری نمونه، ضعیفتر از روش های off-policy هستند. چون اپدیت کردن q-value ها تنها بر اساس نمونه هایی است که توسط همین سیاستی تولید شده است که با آن ارزیابی می شوند. اما روش های off-policy به طور کلی بهره وری بیشتری دارند چون می توانند از تجربه ی سیاست های مختلفی استفاده کنند در نتیجه به طور کلی در تعداد نمونه کمتر، نتیجه بهتری خواهند داشت.

اکتشاف محیط:

در روش های on-policy، یک استراتژی (مقلاً epsilon-greedy) انتخاب شده، اکشن ها با همان استراتژی انتخاب شده و با همان استراتژی نیز ارزیابی می شوند. به همین دلیل اگر exploration کافی نباشد، عامل ممکن است نتواند جفت های s,a بهینه را پیدا کند چون که خود سیاست ارزیابی نیز به خوبی نتوانسته است تمامی s,a ها را به تعداد کافی ببیند. اما در مقابل، روش های off-policy، ارزیابی، ارتباط به سیاست استفاده شده برای تولید نمونه ندارد بنابراین اگر π_b خوب نباشد، سیاست ارزیابی، عامل را مجبور می کند که به اکتشاف ادامه داد و جفت s,a های بهتری را تجربه کند.

مقاومت:

سیاست های on-policy، ممکن است موجب ایجاد واریانس بالا در q-value ها شود. درواقع اگر سیاست فعلی (که همان سیاست ارزیابی نیز است) موجب تجربه و تولید نمونه های نامناسب از محیط باشد،

یادگیری عامل ناپایدار خواهد بود. درواقع الگوریتم در برابر محیط های نویزی از مقاومت خوبی برخوردار نخواهد بود.

اما در off-policy ها، چون روش می تواند از سیاست ها و تجربه های مختلفی بهره گیرد، از پایداری و مقاومت بیشتری برخوردار خواهد بود و اگر هم در برخی از episode ها، تصمیمات غیر بهینه گرفته شود، با استفاده از سیاست اریابی (که از سیاستی که عامل با آن زندگی میکند متفاوت است)، خود را اصلاح کرده و در ادامه سیاست های بهتری را دنبال خواهد کرد.

یک سیستم job recommendation را در نظر بگیرید.

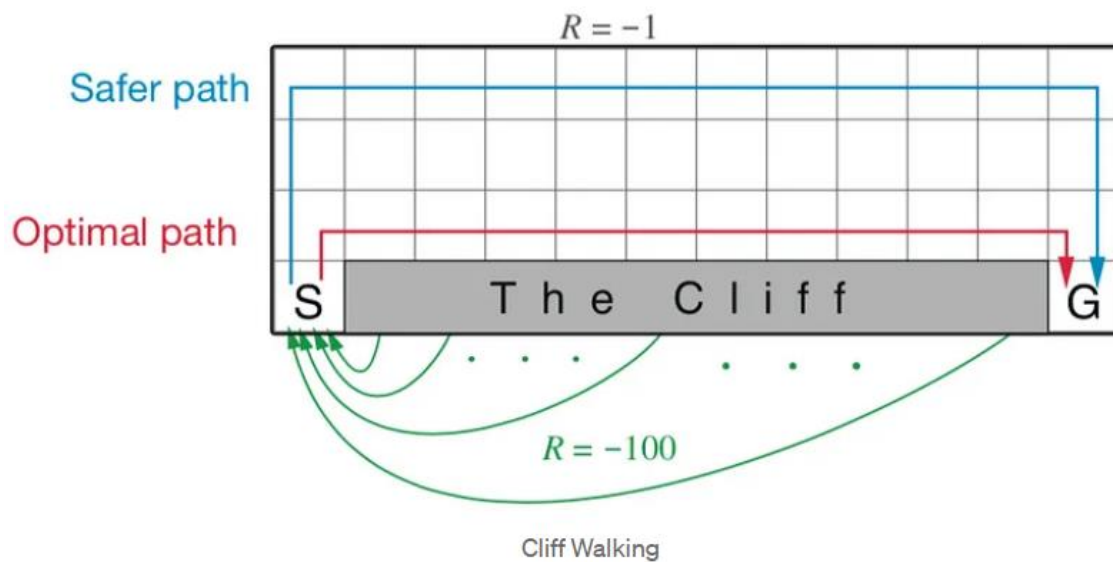
اگر از الگوریتم های on-policy استفاده شود، عامل تنها از سیاستی که همان لحظه در حال اجرا است می تواند یاد بگیرد یعنی از دیتاهای دیگری که در گذر زمان برای اکشن های دیگر بدست آمده است استفاده نمی کند اما الگوریتم های off-policy این امکان را می دهد.

در الگوریتم on-policy، اگر سیاست فعلی خوب نباشد، موجب می شود که الگوریتم On-policy عملکرد ضعیفی داشته باشد و نتواند برخی از (s,a) ها را به تعداد کافی ببیند مثلاً ممکن است به کسی که به دنبال شغل web developer است، هیچ گاه شغلی با عنوان python developer پیشنهاد ندهد درحالیکه ممکن است شخص، کاندید خوبی برای این عنوان شغلی باشد. اما در الگوریتم off-policy، آن s,a ها نیز احتمال دیده شدن خواهند داشت و بدین ترتیب در episode های بعدی نیز انتخاب شده و در نهایت exploration بهتری خواهند داشت.

پاسخ ۴. پیاده‌سازی

الف.

الگوریتم Q-Learning برای محیط Cliff پیاده‌سازی شده [7] و نتایج گزارش خواهند شد. تصویری از الگوریتم cliff در شکل زیر قابل مشاهده است.



شکل ۳. مسیر safe و optimal در محیط cliff

الگوریتم Q-learning یک الگوریتم off-policy است که در آن اپدیت کردن q-value ها به صورت زیر خواهد بود.

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$
Loop for each episode:
 Initialize S
 Loop for each step of episode:
 Choose A from S using policy derived from Q (e.g., ε -greedy)
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
 $S \leftarrow S'$
 until S is terminal

شکل ۴. الگوریتم Q-learning

در این الگوریتم، اکشن فعلی با استفاده از سیاست epsilon-greedy بدست میاید در حالیکه اکشن در $S=S'$ ، اکشنی انتخاب می شود که بیشترین مقدار q-value را به خود اختصاص داده است.

پیاده سازی به این صورت است که هنگامی که در $S=S$ قرار داریم، اکشن با سیاست $\pi_b = \varepsilon_greedy$ و توسط تابع `get_action` بدست میاید. در بخش اپدیت کردن q-value ها نیز `next_max_q` مقدار q-value متناظر با اکشنی است که بیشترین q-value را دارد.

محاسبه ی reward: محیط این مسئله، CliffWalking از کتابخانه ی Gymnasium است. در این مسئله اگر عامل به $S=47$ برسد، اپیزود به پایان میرسد و اگر به صخره برخورد کند، $\text{reward} = -100$ دریافت میکند و در غیر اینصورت $\text{reward} = -1$ ایجاد می شود (این بخش درون خود کتابخانه ی gymnasium پیاده سازی شده است). در هر اپیزود، عامل انقدر در محیط حرکت می کند تا به $S=47$ یعنی `location[3,11]` برسد و فلگ `terminate=True` شود. در غیر اینصورت در محیط حرکت کرده و ممکن است چندین بار به صخره برخورد کند و چندین بار $\text{reward} = -100$ دریافت کند.

محاسبه ی regret: برای محاسبه ی regret، باید بیشترین reward ممکن را بدانیم. در این محیط، `max_reward` برابر است با 13-. زیرا بین نقطه شروع و نقطه `terminate`، 13 استیت وجود دارد. بنابراین در هر اپیزود، میزان regret برابر خواهد بود با:

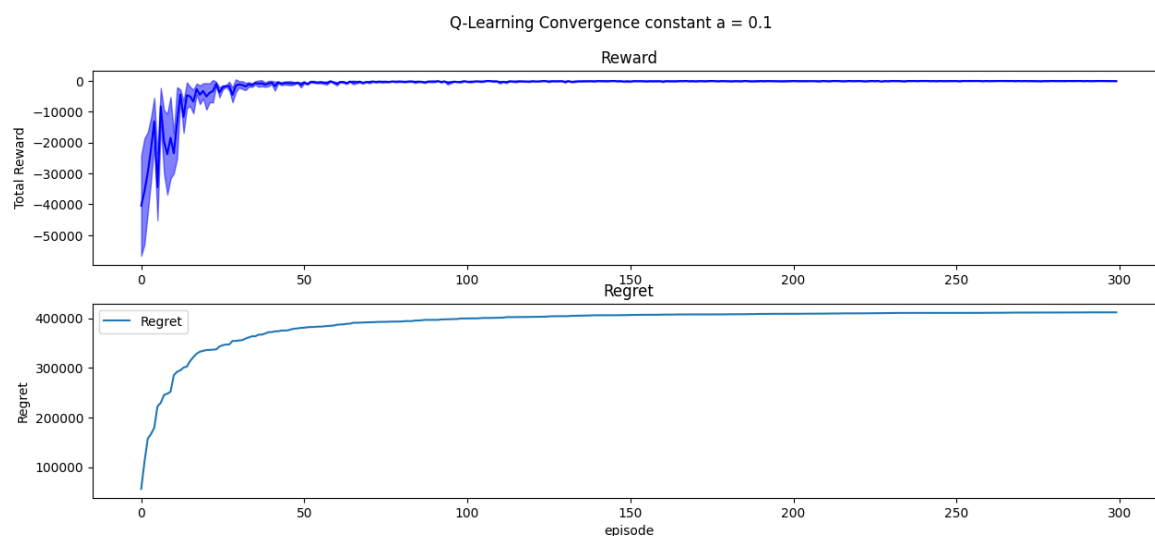
$$\text{regret} = -13 - \text{reward}_{\text{total}}$$

این محاسبه، پس از truncate شدن و یا terminate شدن عامل انجام می شود.

حالت اول: $\alpha = 0.1$

در حالت اول، $\alpha = 0.1$ در نظر گرفته شده و همچنین مقدار ϵ در ابتدا مقدار ۱ در نظر گرفته شده (تا exploration خوبی داشته باشیم) و در هر تکرار در ضریب 0.95 ضرب می شود تا به آرامی به سمت greedy حرکت کنیم. البته کمترین مقدار ϵ برابر با 0.1 در نظر گرفته شده است تا همواره exploration را در نظر داشته باشیم.

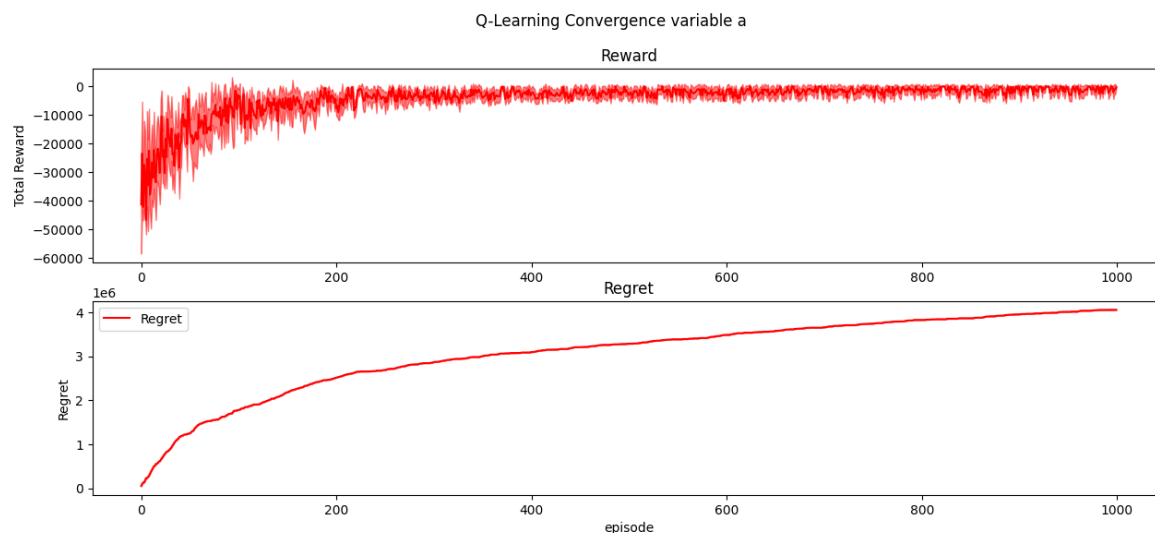
الگوریتم ۵ بار اجرا شده و نتایج (نمودار reward و regret به همراه confidence interval) در ادامه قابل مشاهده هستند.



شکل ۵. نمودار reward و regret الگوریتم Q-learning با $\alpha = 0.1$ ثابت

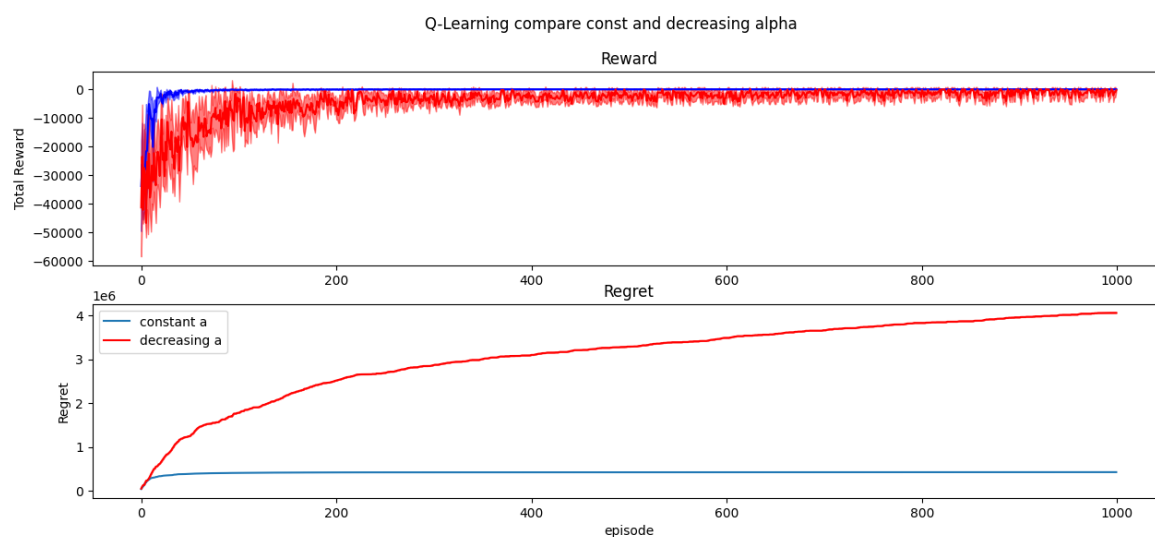
حالت دوم: α متغیر و کاهشی

در این حالت، مقدار α در هر step در ضریب 0.9 ضرب شده است.



شکل ۶. نمودار **reward** و **regret** الگوریتم **Q-learning** با α کاهشی

در شکل زیر، مقایسه α ثابت و متغیر قابل مشاهده است.



شکل ۷. مقایسه ی نتایج α ثابت و متغیر در الگوریتم **Q-Learning**

مشاهده می شود که میزان **regret** در حالتی که α مقدار ثابت در نظر گرفته شده است بسیار کمتر از حالت کاهنده است. همچنین **reward** ها نیز بسیار زودتر به عدد 13- همگرا شده اند.

سرعت اجرای کد در حالتی که α ثابت در نظر گرفته شده است، بسیار بیشتر از حالتی بود که α متغیر و به صورت کاهنده در نظر گرفته شده بود.

ب.

در این بخش، الگوریتم های SARSA و n-step tree پیاده سازی شده و نتایج گزارش خواهند شد.

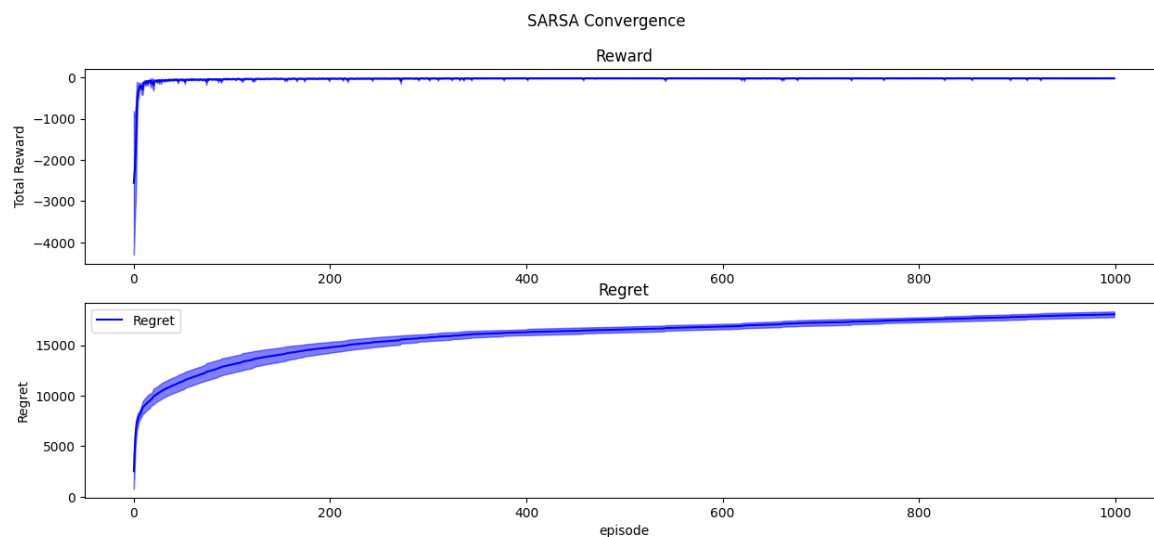
الگوریتم SARSA: [6] [8]

این الگوریتم بسیار شبیه به Q-learning است. با این تفاوت که الگوریتم SARSA، یک الگوریتم On-policy است. یعنی سیاستی که با آن ارزیابی میکنیم، همان سیاستی است که عامل با آن در محیط زندگی کرده است.

در الگوریتم SARSA، عامل ترجیح داده است که از مسیر safe حرکت کند بنابراین مقدار $\text{regret} = -17$ در نظر گرفته شده است. (در شکل بنظر میرسید -۱۳ باشد اما چون $\text{reward} = -100$ بسیار عدد منفی بدی است، عامل ترجیح میدهد از مسیر دورتری حرکت کند و پاداش 17- دریافت کند اما به صخره نزدیک نباشد)

در الگوریتم SARSA، هنگامی که در $S=S'$ قرار داریم، اکشن بعدی را با توجه به سیاست $\epsilon - greedy$ انتخاب خواهیم کرد. در حالیکه در الگوریتم Q-learning، این اکشن، اکشنی بود که بیشترین q-value را به خود اختصاص داده بود ($\text{argmax}(q(s'))$). الگوریتم اپدیت کردن q-value ها در sarsa در شکل ۱ (که برگرفته از کتاب Sutton [5] است) نمایش داده شده است. در این الگوریتم، مقدار اولیه $\epsilon = 0.5$ در نظر گرفته شده و به تدریج و با ضریب 0.9 کاهش میابد.

نتایج پیاده سازی الگوریتم SARSA در شکل ۸ قابل مشاهده است.

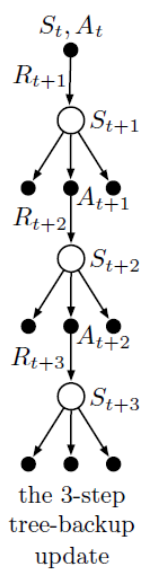


شکل ۸. نمودار **reward** و **regret** الگوریتم **SARSA**

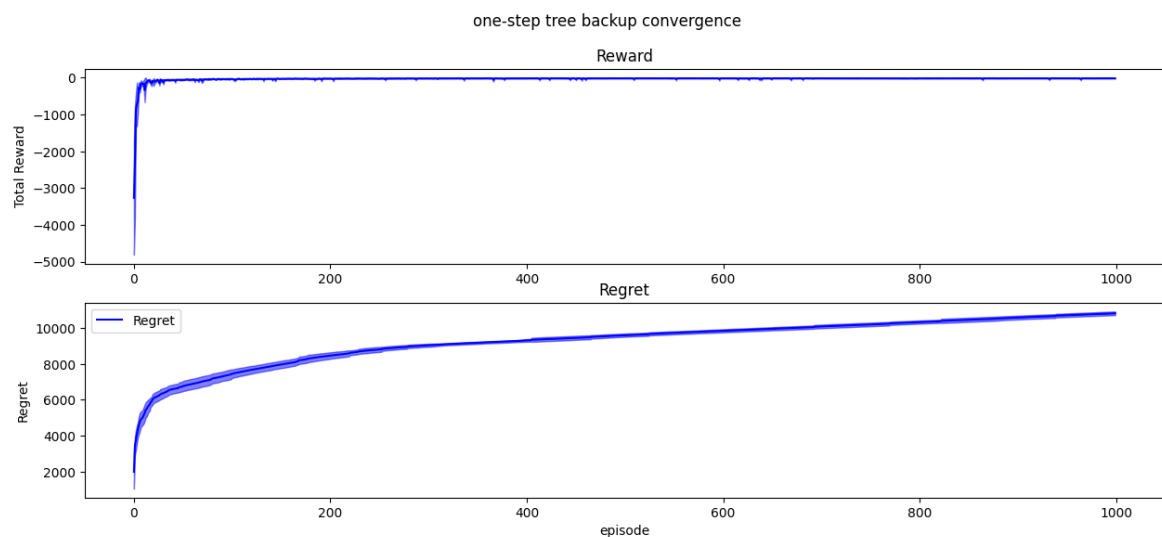
همانطور که در شکل مشاهده می شود، سیاست، epsilon-greedy است و از انجایی که مقدار ϵ را هیچ گاه صفر نکرده ایم، مقدار regret صعود خواهد ماند. اگر مقدار $\epsilon_{\min} = 0$ در نظر گرفته شود (یعنی در نهایت به سیاست کاملاً greedy) همگرا شویم، نمودار regret ثابت خواهد ماند.

الگوریتم **one-step tree backup**: [6]

الگوریتم پیاده سازی شده و نتایج در زیر قابل مشاهده است.

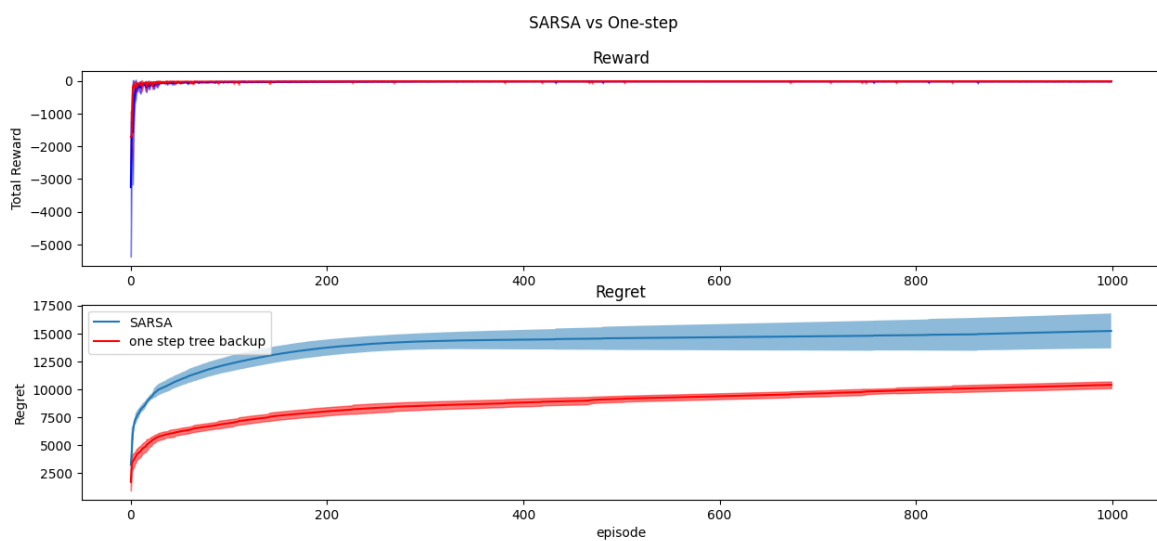


شکل ۹. شمایی از الگوریتم **N-step tree**

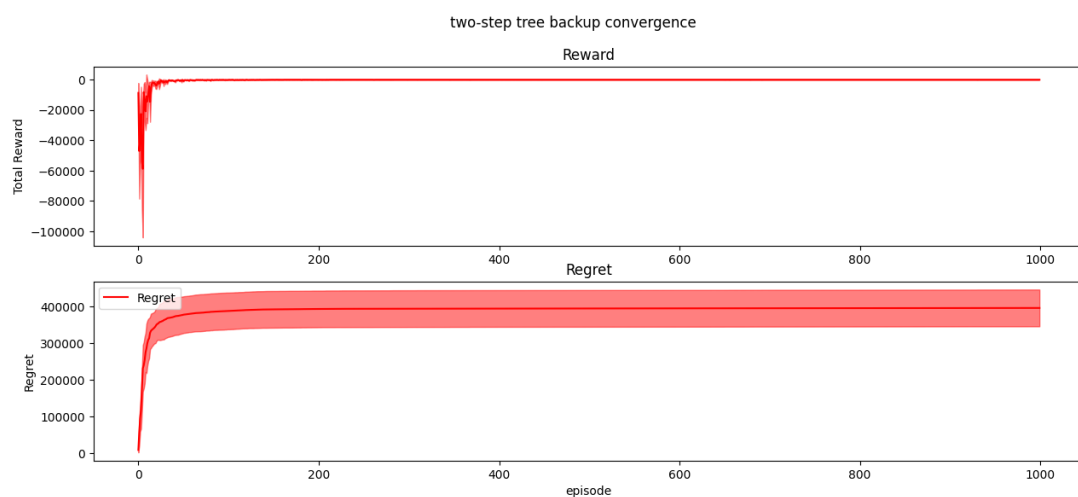


شکل ۱۰. نمودار **reward** و **regret** الگوریتم **one-step**

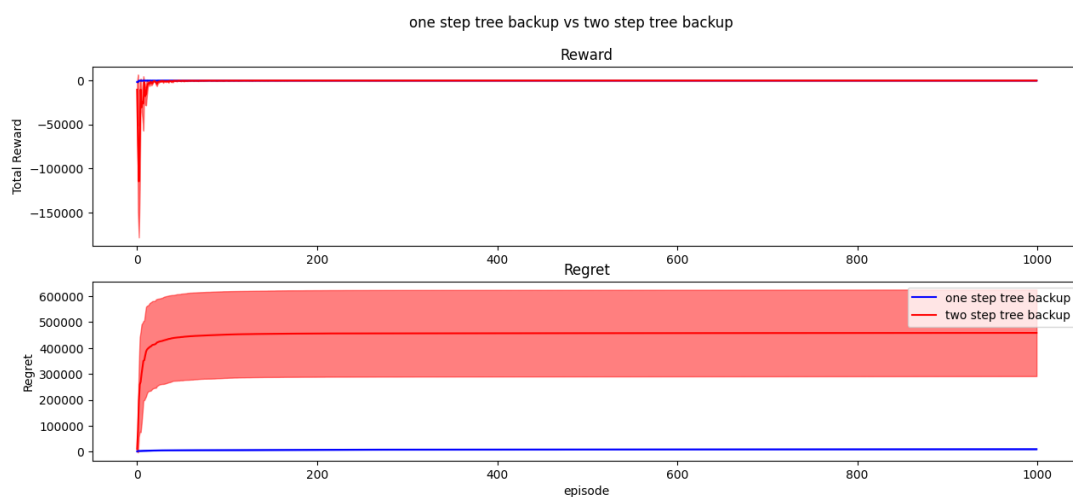
مقایسه الگوریتم های sarsa و one step tree backup در تصویر ۱۱ قابل مشاهده است.



شکل ۱۱. مقایسه الگوریتم های **SARSA** و **one-step backup**



شکل ۱۲. نمودار **reward** و **regret** الگوریتم **two-step**



شکل ۱۳. مقایسه الگوریتم های **one step backup** و **two-step backup**



شکل ۱۴. نمودار **reward** و **regret** الگوریتم **three-step**



شکل ۱۵. مقایسه الگوریتم های **three-step backup** و **two step backup**

ج

در این بخش الگوریتم off-Policy Monte Carlo پیاده سازی می شود. در این الگوریتم، سیاستی که با آن در محیط زندگی میکنیم، *soft* است (مثلاً رندوم یا *epsilon-greedy*) اما سیاستی که آن را ارزیابی کرده و بهبود می دهیم، *deterministic* است (یعنی *greedy*). برای همگرایی لازم است که هر جفت s, a بینهایت بار دیده شود (که البته در پیاده سازی لازم نیست بینهایت بار *loop* بزنیم). برای این مورد، باید سیاست *soft* باشد تا احتمال انتخاب هر s, a وجود داشته باشد.

الگوریتم off-policy monte carlo به صورت شکل ۱۳ قابل مشاهده است.

Off-policy MC control, for estimating $\pi \approx \pi_*$

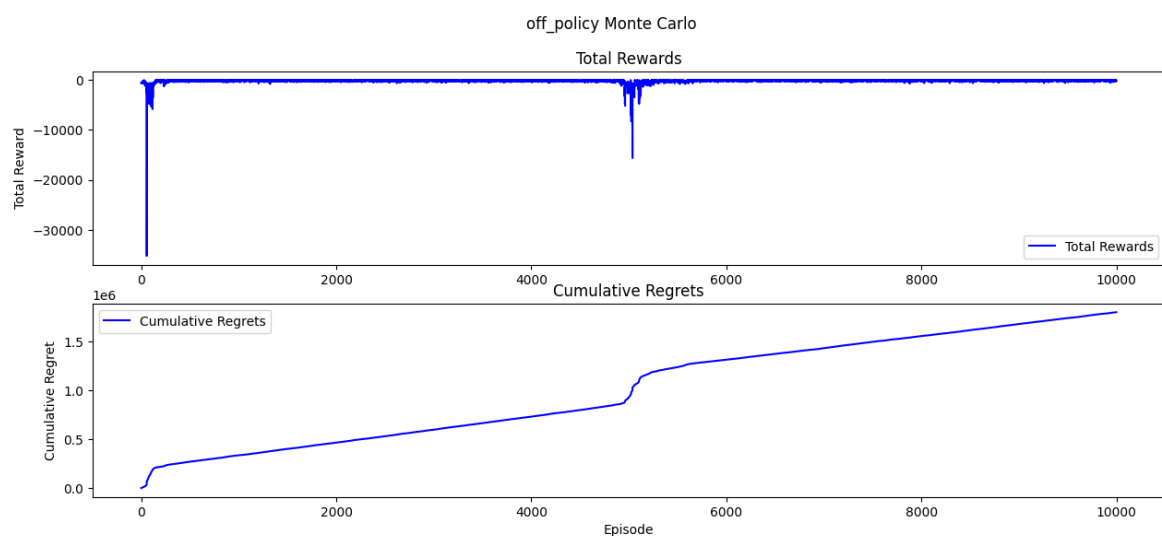
```
Initialize, for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$ :  
   $Q(s, a) \in \mathbb{R}$  (arbitrarily)  
   $C(s, a) \leftarrow 0$   
   $\pi(s) \leftarrow \operatorname{argmax}_a Q(s, a)$  (with ties broken consistently)  
  
Loop forever (for each episode):  
   $b \leftarrow$  any soft policy  
  Generate an episode using  $b$ :  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$   
   $G \leftarrow 0$   
   $W \leftarrow 1$   
  Loop for each step of episode,  $t = T-1, T-2, \dots, 0$ :  
     $G \leftarrow \gamma G + R_{t+1}$   
     $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$   
     $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$   
     $\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$  (with ties broken consistently)  
    If  $A_t \neq \pi(S_t)$  then exit inner Loop (proceed to next episode)  
     $W \leftarrow W \frac{1}{b(A_t|S_t)}$ 
```

شکل ۱۶. الگوریتم [5] Off-policy monte carlo

الگوریتم پیاده سازی شده [9] و نتایج به صورت زیر قابل مشاهده است.

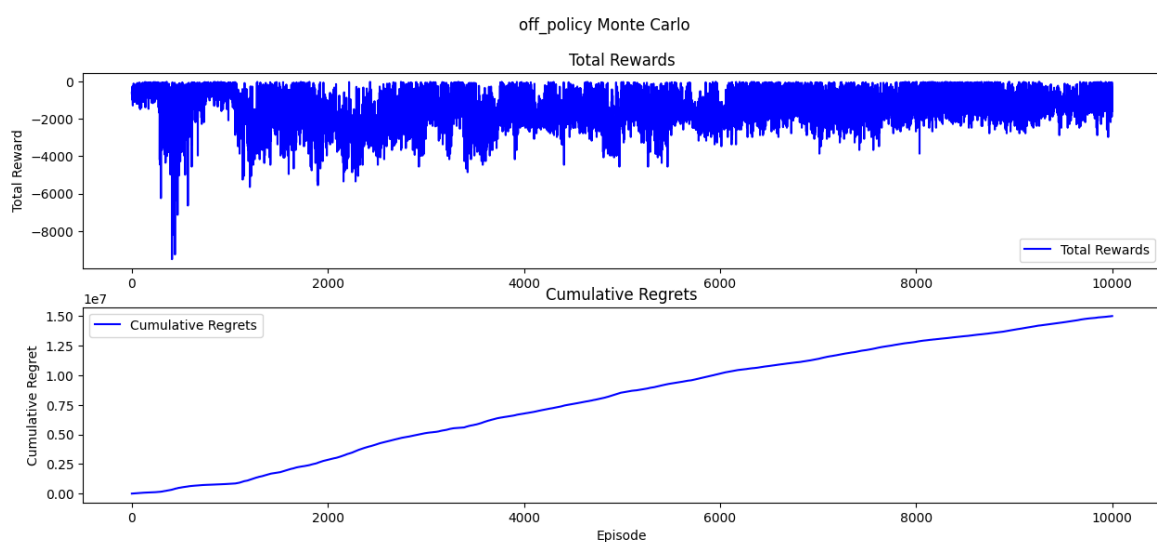
در الگوریتم off-policy monte carlo، سیاستی که با آن زندگی می کنیم، epsilon-greedy بوده و سیاست هدف، $\arg \max(q_value)$ ها است. چون این دو سیاست با یکدیگر متفاوت هستند، off-policy است.

الگوریتم در دو حالت ϵ ثابت و ϵ کاهشی در behavior policy پیاده سازی و نتایج آن در زیر قابل مشاهده است. در حالت ϵ کاهشی، مقدار اولیه $\epsilon = 0.5$ در نظر گرفته شده و با ضریب 0.9 کاهش یافته است.

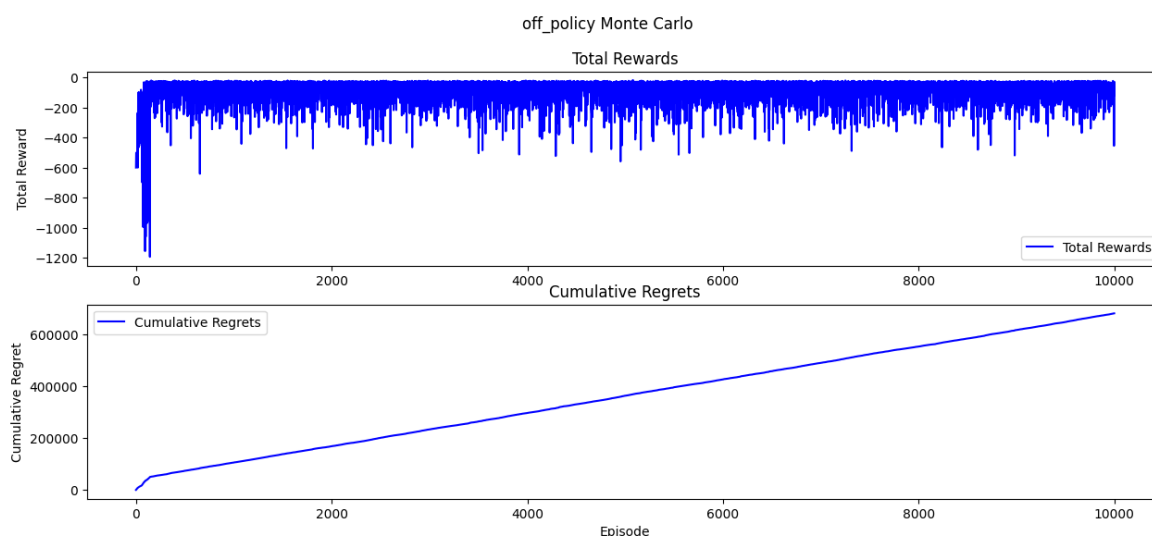


شکل ۱۷. نمودار reward و regret الگوریتم off-policy monte carlo در حالت ϵ کاهشی

دو تصویر زیر مربوط به همین الگوریتم اما در حالت ϵ ثابت هستند.



شکل ۱۸. نمودار reward و regret الگوریتم off-policy monte carlo در حالت $\epsilon = 0.5$ ثابت



شکل ۱۹. نمودار **reward** و **regret** الگوریتم **off-policy monte carlo** در حالت $\epsilon = 0.1$ ثابت

همانطور که در تصویر مشاهده می شود، این الگوریتم در هیچکدام از حالت های بررسی شده، همگرایی مناسبی نداشته است. [6]

مراجع:

[1] GPT 4-o, Prompt: “consider an MDP that the agent is interacting with human and im using Q learning since the rewards are sparse, what effect does it have on the convergence rate?”

[2] Inspired by GPT 4-o, Prompt: “what is the effect of sparse reward on the q learning”

[3] GPT 4-o, Prompt: “what are other problems associated with rl agents interacting with human?”

[4] GPT 4-o, Prompt: “compare sarsa and expected sarsa in terms of convergance and robustness?”, “in which condition sarsa is better than expected sarsa”

[5] Sutton Book

[6] Inspired by GPT 4-o

[7] inspired by Handson-3 Q-learning

[8] inspired by Handson-3 Sarsa

